

STM32 Bare-Metal Drivers & HEXBOOT Roadmap

Vision

This repository documents a structured journey from STM32 bare-metal driver development to advanced bootloader architecture, system design, RTOS, embedded Linux, FPGA prototyping, and eventually ASIC development.

STM32F030 Learning Path

Foundation Layer

Project 01 – GPIO

- Register-level GPIO control
- Input/Output configuration

Project 02 – USART

- UART communication
- Baud rate generation

Project 03 – Timers

- Basic timer configuration
- Delay generation

Project 04 – Interrupts

- NVIC fundamentals
- ISR implementation

Project 05 – Communication Drivers

- Driver abstraction improvements
- Reusable peripheral interfaces

Project 06 – Flash Driver

- Flash erase
- Flash programming
- Flash verification

Project 07 – CRC

- CRC peripheral usage
- Data integrity verification

Project 08 – Firmware Reception

- UART packet reception
- Firmware buffering concepts

Project 09 – Metadata Framework

- Firmware metadata structure
- Flash storage design

Project 10 – Firmware Validation

- Application validation
- Metadata verification
- Safe application jump

Project 11 – Simple IAP

- UART firmware update
- CRC verification
- Metadata storage
- Automatic application jump

HEXBOOT Evolution

Project 12 – Streaming IAP

Objective

Remove firmware-size dependency on RAM.

Architecture

Receive Chunk ↓ Update CRC ↓ Program Flash ↓ Receive Next Chunk

Key Learning

- Streaming architecture
 - Memory optimization
 - Large firmware support
-

Project 13 – Interrupt Driven IAP

Objective

Replace polling-based firmware reception with interrupt-driven reception.

Key Learning

- USART interrupts
 - Event-driven systems
 - ISR design
 - Firmware reception pipeline
-

Project 14 – State Machine IAP

Objective

Transform the bootloader into a state-machine driven architecture.

Example States

WAIT_HEADER ↓ WAIT_LENGTH ↓ WAIT_PAYLOAD ↓ WAIT_CRC ↓ VERIFY ↓ PROGRAM_FLASH ↓
JUMP_APPLICATION

Key Learning

- Finite State Machines
 - Protocol processing
 - Embedded system architecture
-

Project 15 – Ping-Pong Buffer IAP

Objective

Implement double-buffer firmware reception.

Architecture

Buffer A Receiving ↓ Buffer B Programming

Buffer B Receiving ↓ Buffer A Programming

Key Learning

- Producer-consumer design
 - Double buffering
 - Concurrent data flow
-

Project 16 – DMA Laboratory

Experiments

- DMA Memory-to-Memory
- DMA UART RX
- DMA UART TX
- DMA Circular Buffer

Key Learning

- DMA architecture
 - Hardware-assisted transfers
 - High-performance communication
-

Project 17 – Packet Framing Protocol

Objective

Design a robust firmware transport protocol.

Current Protocol

Header Length Payload CRC

Future Protocol

SOF Type Sequence Length Payload CRC EOF

Key Learning

- Protocol design
 - Packet framing
 - Sequence tracking
 - Error recovery
-

Project 18 – HEXBOOT Framework Cleanup

Objective

Refactor the entire bootloader into a reusable framework.

Proposed Structure

bootloader/ protocol/ flash/ crc/ metadata/ uart/ dma/ state_machine/

Key Learning

- Software architecture
 - Reusability
 - Maintainability
 - Portfolio-quality code
-

STM32F0 Graduation Criteria

At the completion of Project 18:

- ✓ Flash Driver
 - ✓ CRC Driver
 - ✓ Metadata System
 - ✓ Firmware Validation
 - ✓ Streaming IAP
 - ✓ Interrupt Driven IAP
 - ✓ State Machine IAP
 - ✓ Ping-Pong Buffer IAP
 - ✓ DMA
 - ✓ Packet Protocol
 - ✓ Bootloader Framework
-

STM32H723 Roadmap

H7-01 Architecture Explorer

Topics

- Cortex-M7 Architecture
- Cache
- Bus Matrix
- ITCM
- DTCM
- AXI SRAM
- Clock Tree

Objective

Understand how a high-performance MCU is architected internally.

H7-02 Minimal Platform Bring-Up

Features

- Custom Clock Configuration
- GPIO
- USART
- SysTick

Objective

Build a complete H723 bare-metal platform from scratch.

Future Roadmap

STM32H7 ↓ RTOS ↓ Advanced Bootloader ↓ NXP FRDM i.MX 8M Plus ↓ Embedded Linux ↓ FPGA Prototyping ↓ Custom RISC-V Experiments ↓ ASIC Fundamentals ↓ Silicon Architecture

Personal Motto

"I will climb all the mountain in my path to success."